1. Mục tiêu

- Hiểu rõ và có khả năng áp dụng ba nguyên lý còn lại của SOLID: **Liskov Substitution Principle (LSP)**, **Interface Segregation Principle (ISP)**, và **Dependency Inversion Principle (DIP)**.
- Nắm vững vai trò của Interface, kế thừa, và Dependency Injection trong thiết kế phần mềm.
- Chuẩn bị tốt cho phần thuyết trình và demo.

2. Nội dung cần nghiên cứu và chuẩn bị

2.1. Liskov Substitution Principle (LSP)

- **Khái niệm:** Các đối tượng của lớp con phải có thể thay thế được các đối tượng của lớp cha mà không làm thay đổi tính đúng đắn của chương trình.
- **Tầm quan trọng:** Đảm bảo tính nhất quán và đáng tin cậy của hệ thống khi sử dụng kế thừa và đa hình.
- **Ví dụ:**
  - **Vi phạm LSP:** Một class `Square` kế thừa từ `Rectangle`, nhưng việc set width và height của `Square` có hành vi khác với `Rectangle` (vì hình vuông có width = height).
  - **Áp dụng LSP:** Thiết kế lại hierarchy class hoặc sử dụng interface để đảm bảo tính thay thế được.
- **Chuẩn bị demo:** Viết code minh họa trường hợp vi phạm LSP và cách khắc phục.

2.2. Interface Segregation Principle (ISP)

- **Khái niệm:** Không nên ép buộc client phụ thuộc vào các interface mà chúng không sử dụng. Nên tách các interface lớn thành nhiều interface nhỏ, chuyên biệt.
- **Tầm quan trọng:** Giảm sự phụ thuộc không cần thiết, làm cho code linh hoạt và dễ bảo trì hơn.
- **Ví dụ:**
  - **Vi phạm ISP:** Một interface `IWorker` có cả method `work()` và `eat()`. Class `Robot` implement `IWorker` nhưng không cần method `eat()`.
  - **Áp dụng ISP:** Tách thành `IWorkable` (có `work()`) và `IFeedable` (có `eat()`). Class `Robot` chỉ implement `IWorkable`.
- **Chuẩn bị demo:** Viết code minh họa việc tách interface và lợi ích của nó.

2.3. Dependency Inversion Principle (DIP)

- **Khái niệm:**
  - Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào abstraction.
  - Abstraction không nên phụ thuộc vào chi tiết. Chi tiết nên phụ thuộc vào abstraction.
- **Tầm quan trọng:** Giảm sự kết nối chặt chẽ giữa các module, làm cho hệ thống linh hoạt và dễ test.
- **Ví dụ:**
  - **Vi phạm DIP:** Class `OrderService` trực tiếp tạo instance của `MySQLDatabase` trong constructor.
  - **Áp dụng DIP:** `OrderService` nhận một interface `IDatabase` thông qua constructor (Dependency Injection). Việc quyết định sử dụng `MySQLDatabase` hay `PostgreSQLDatabase` được thực hiện ở nơi khác.
- **Chuẩn bị demo:** Viết code minh họa Dependency Injection và lợi ích của nó.

3. Vai trò của Interface, kế thừa, và Dependency Injection

- **Interface:** Định nghĩa contract mà các class phải tuân theo, giúp tách biệt abstraction và implementation.
- **Kế thừa:** Cho phép tái sử dụng code và tạo ra hierarchy class, nhưng cần tuân theo LSP.
- **Dependency Injection:** Kỹ thuật để implement DIP, giúp giảm sự phụ thuộc và tăng tính linh hoạt của hệ thống.

4. Chuẩn bị thuyết trình và demo

- **Slide:** Chuẩn bị slide thuyết trình rõ ràng, tập trung vào từng nguyên lý với ví dụ minh họa cụ thể.
- **Code Demo:** Đảm bảo code demo chạy được và minh họa đúng từng nguyên lý. Chuẩn bị kịch bản demo từng bước.
- **Thời gian:** Phân bổ thời gian hợp lý cho cả 3 nguyên lý (khoảng 10-15 phút mỗi nguyên lý).

5. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu sâu về LSP	TV1	2
Nghiên cứu sâu về ISP	TV2	2
Nghiên cứu sâu về DIP	TV3	2
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	3
Viết code demo LSP (có/không áp dụng)	TV1	3
Viết code demo ISP (có/không áp dụng)	TV2	3
Viết code demo DIP (có/không áp dụng)	TV3	3
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**21**

6. Tài liệu tham khảo

- **Sách:** "Clean Architecture" của Robert C. Martin (Uncle Bob) - Chương về SOLID Principles.
- **Bài viết:** Tìm kiếm các bài viết chuyên sâu về LSP, ISP, DIP và Dependency Injection.
- **Video:** Các video hướng dẫn trên YouTube về SOLID principles, đặc biệt là 3 nguyên lý cuối.

7. Ghi chú quan trọng

- **Hiểu rõ mối liên hệ:** Các nguyên lý SOLID có mối liên hệ chặt chẽ với nhau. Hãy cố gắng hiểu cách chúng bổ trợ cho nhau.
- **Thực hành nhiều:** Chỉ có thực hành nhiều mới giúp bạn nắm vững các nguyên lý này.
- **Đặt câu hỏi:** Đừng ngần ngại đặt câu hỏi nếu có bất kỳ điểm nào chưa rõ.

Chúc các bạn học tập hiệu quả và thành công!